

GEval — RAG Application Quality Evaluation (Concept-Clear Notes)

LLM-as-a-Judge ko reliable banane ka tareeka — Correctness, Completeness & Style metrics

1. Kahan Hum Abhi Hain? (Recap)

RAG evaluation ek offline eval suite ke through teen levels par prepare hoti hai:

- Component level** — individual pipeline parts (retriever, generator) ka eval — **[Done]**
- Pipeline level** — poori RAG pipeline ka eval (Recall, Precision, Faithfulness, Answer/Context Relevance) — **[Done]**
- Application level** — Quality, Safety, Operations — **[Aaj: Quality]**

Application Quality ke andar teen metrics cover honge: **Correctness** (kya answer sahi hai), **Completeness** (kya sab sub-questions cover ho rahe hain), aur **Style** (kya answering style brand/teaching style match karta hai).

2. Count-Based Metrics vs Judgment-Based Metrics

Ab tak jo 5 metrics padhe the — **Recall, Precision, Faithfulness, Answer Relevance, Context Relevance** — un sab mein ek common pattern tha: unhe **count-based** tareeke se calculate kiya jata tha.

Count-Based Approach Kaise Kaam Karta Hai

- Generated answer ko LLM se **claims** mein break karo
- Har claim ko check karo — context/expected answer mein exist karta hai ya nahi
- Favorable claims ka ratio nikaal ke final score banao (e.g. Faithfulness = 3/4)

Problem: Kuch metrics jaise Style ya Correctness mein counting ka concept hi kaam nahi karta — kyunki style ya correctness sentence-level pe nahi, poore **answer-level** pe judge hota hai (e.g. ek analogy isolation mein context se match nahi karegi, lekin poore answer ke context mein valid hoti hai). Yahan par aapko **judgment** chahiye hota hai — kisi ko poora answer padh ke 0–10 ka score dena hota hai.

3. Naive "LLM-as-a-Judge" ka Problem

Simple approach: ek Golden Dataset (Question + Correct Answer) banao, phir LLM judge ko Question, Expected Answer aur Actual Answer dekar bolo — "0 se 10 ke beech score do".

Yeh approach theoretically kaam karti hai, lekin practically **reliable nahi hai** — kyunki score ka variance bahut high hota hai. Same input pe baar-baar run karo to score kabhi 6, kabhi 7, kabhi 8 aa sakta hai.

Do Reasons Hain Iske Peeche

- Loose criteria:** Sirf ek high-level line di gayi hai ("factually correct hai ya nahi decide karo") — har LLM call mein model isko thoda alag interpret kar sakta hai
- Direct token output:** Model seedha ek integer bol raha hai (7, 8, ya 9) — internally token probabilities close hoti hain (e.g. "8" ko 51%, "7" ko 40%), toh alag runs mein alag token pick ho sakta hai

4. GEval Kya Hai?

GEval ek research paper hai (2023) jo LLM-as-a-Judge ko zyada **deterministic aur reliable** banane ka tareeka propose karta hai. GEval sirf do naye innovations laata hai:

- Innovation 1:** High-level criteria ko Chain-of-Thought (CoT) se detailed **Evaluation Steps** (ek fixed rulebook) mein todna
- Innovation 2:** Final token output lene ke bajaye, **top-k tokens ki log probabilities** ka weighted average lena

GEval Process — 5 Steps

Step	Kya Hota Hai
1. Specify the metric	Metric ka naam (e.g. Correctness) aur high-level criteria define karo
2. Lock evaluation steps	CoT se criteria ko fixed rubric/steps mein break karo — har run mein same
3. Build judge prompt	Evaluation steps + scoring rubric + input/expected/actual fields se ek system prompt banta hai
4. Probability-weighted score	Top-k score tokens (jaise 6,7,8,9) ki probabilities nikaal ke weighted average lo
5. Normalize, threshold, explain	Score ko 0–1 range mein normalize karo, threshold se pass/fail decide karo, reason bhi milta hai

5. Probability-Weighted Scoring — Worked Example

Jab LLM judge score generate karta hai, autoregressive tareeke se har possible token (0–10, digits, words) ko ek probability assign hoti hai. Normally sirf highest probability wala token output print hota hai — GEval iske bajaye top-k tokens ki probabilities ka **weighted average** nikalta hai.

Step A — Top Tokens Aur Unki Raw Probabilities

Token	Probability
"8"	0.70
"7"	0.20
"9"	0.05
"the"	0.01
".."	0.01

Non-numerical tokens ("the", ":",") ignore kar diye jaate hain — sirf numerical scores matter karte hain.

Step B — Sirf Numerical Tokens Rakho: 8, 7, 9

Score	Probability
8	0.70
7	0.20
9	0.05

Step C — Normalize Karo (sum = 0.95)

$8 \rightarrow 0.70 / 0.95 = \mathbf{0.7368}$ | $7 \rightarrow 0.20 / 0.95 = \mathbf{0.2105}$ | $9 \rightarrow 0.05 / 0.95 = \mathbf{0.0526}$

Step D — Weighted Sum Nikalo (score × normalized probability)

$8 \times 0.7368 = 5.8947$
 $7 \times 0.2105 = 1.4737$
 $9 \times 0.0526 = 0.4737$
 $\text{Sum} = 5.8947 + 1.4737 + 0.4737 = \mathbf{7.84}$

Final normalize (+10): $7.84 / 10 = \mathbf{0.784}$ — Yehi final GEval score hai jo threshold (e.g. 0.7) se compare hoke pass/fail decide karta hai.

"Agar simple approach use karte (sirf max-probability token '8' le lete), to output seedha 8 hota. Weighted average approach zyada stable hai kyunki agli baar bhi score 7.84 ke aas-paas hi rahega (7.4–7.9), 6 se 8 tak jump nahi karega."

6. DeepEval Library mein GEval Implementation

DeepEval library GEval ka pooraa implementation provide karti hai — hume manually kuch calculate nahi karna padta.

Judge System Prompt (Template)

```
You are an evaluator scoring the correctness of an AI-generated answer.
You will judge the ACTUAL OUTPUT against the EXPECTED OUTPUT using the
evaluation steps below.

Evaluation steps:
1. Compare only the factual claims in the actual output against the expected
   output.
2. A claim is wrong only if it CONTRADICTS the expected output or is factually
   false.
3. A factually accurate answer scores high even if it is shorter or covers
   fewer points.
4. Do NOT deduct for brevity or omitted points — only wrong statements count.
5. Additional correct information must never lower the score.

Scoring rubric (0-10):
9-10: All stated claims correct and consistent. No contradictions. Brevity is
      fine.
5-8:  Mostly correct but one minor inaccuracy.
0-4:  Contains a clear factual error or a claim that contradicts the expected
      output.
```

DeepEval Code — Correctness Metric

```
from deepeval import evaluate
from deepeval.test_case import LLMTestCase, LLMTestCaseParams
from deepeval.metrics import GEval

correctness = GEval(
    name="Correctness",
    criteria="Determine whether the actual output is factually correct based on the expected output.",
    evaluation_params=[
        LLMTestCaseParams.INPUT,
        LLMTestCaseParams.ACTUAL_OUTPUT,
        LLMTestCaseParams.EXPECTED_OUTPUT,
    ],
    model="gpt-4o-mini",
    threshold=0.7,
)

test_case = LLMTestCase(
    input="What is offline eval?",
    actual_output="Offline eval runs before deployment.",
    expected_output="Offline eval is evaluation done before deployment using a fixed golden dataset.",
)

evaluate(test_cases=[test_case], metrics=[correctness])
```

Code Mein Kya-Kya Pass Karte Hain

- **name** — metric ka naam (Correctness / Completeness / Style)
- **criteria** ya **evaluation_steps** — high-level criteria (CoT khud steps banayega) YA khud se fixed evaluation steps (zyada deterministic)
- **evaluation_params** — kaunse fields judge karne hain (Input, Actual Output, Expected Output)
- **model** — judge LLM (best results GPT-4 class models ke saath)
- **threshold** — kis score ke upar "pass" mana jayega
- **strict_mode=False** — probability-weighted calculation enable rehta hai (True karne se seedha raw token le liya jayega, weighting skip ho jayegi)

7. Criteria Dena vs Khud Evaluation Steps Dena

High-Level Criteria Dena	Khud Evaluation Steps Dena
GEval khud CoT se steps generate karta hai	Aap khud fixed steps provide karte ho
Har LLM call mein steps thoda vary ho sakte hain	Har call mein exact same steps — zyada deterministic
Achha hai jab pipeline design karna abhi shuru kiya ho	Achha hai jab aapko already samajh aa gaya ho ki eval kaise ho raha hai

Rule of thumb: Shuru mein criteria use karo (LLM par trust karo). Jab 2-3 baar chala ke pattern samajh aa jaaye (kaunse tarah ke questions fail ho rahe hain), tab apne khud ke evaluation_steps aur rubric define karo — isse determinism badh jaata hai.

8. Application Quality Pipeline Banana — 3 Metrics

Workflow: Golden Dataset (15 Q&A pairs, human expert ne banaya) → RAG pipeline se actual answer generate karo → GEval metric ke through score karo → results padh ke prompt/rubric improve karo.

A. Correctness

Correctness check karta hai ki answer duniya ki nazar mein (Google-level) factually sahi hai ya nahi — yeh **Faithfulness** se alag hai (jo sirf context-groundedness check karta hai).

Run	Setup	Score	Pass/Fail
1st run	Generic evaluation steps	66%	8 pass / 7 fail
2nd run	Rubric refined — brevity/omission ko penalize mat karo	84%	14 pass / 1 fail
Re-run (stability check)	Same rubric, dobara run	83%	14 pass / 1 fail (same case)

Fix: Failure reasons padhne se pata chala ki generated answers golden answers jitne detailed nahi the (par khud mein correct the). Rubric mein add kiya — "Do NOT penalize for brevity or omitted points" aur scoring bands (9-10 / 5-8 / 0-4) explicitly define kiye. Result: **66% → 84%**.

B. Completeness

Completeness check karta hai ki agar ideal answer mein multiple points (A, B, C) hain, to generated answer un sab points ko cover kar raha hai ya sirf kuch (partial answer).

Run	Setup	Score	Pass/Fail
1st run	Generator prompt sirf 'concise answer' bolta tha	68%	5 pass / 10 fail
2nd run	Generator prompt fix — 'address every part of the question'	75%	14 pass / 1 fail

Root cause: Generator ka system prompt already "concise answers" ke liye restrict kiya hua tha. Fix generator ke prompt mein hua, eval rubric mein nahi.

C. Style

Style check karta hai ki generated answer brand/teaching style ("Campus X" style) follow karta hai ya nahi. Isme koi golden/ideal answer ki zaroorat nahi — sirf ek clear rubric chahiye jo style define kare.

Run	Setup	Score
1st run	Generator ko style ke baare mein kuch nahi bataya gaya	54%
2nd run	Generator prompt mein conversational tone add kiya	74%

Overcorrection warning: Ek run mein rubric mein likha tha "reward analogies/examples" — isse judge har answer mein analogy expect karne laga, jo galat tha. Fix: rubric mein clarify kiya — "analogy is a bonus when concept is abstract, but a clear direct answer is fully acceptable."

9. Key Takeaways

- 5 purane metrics (Recall, Precision, Faithfulness, Answer/Context Relevance) **count-based** the — claims break karke ratio nikalte the
- Style, Correctness, Completeness, Helpfulness jaise metrics **judgment-based** hote hain — inhe count se nahi naapa ja sakta
- Naive LLM-as-a-judge ("0-10 score do") **unreliable** hai — high variance ki wajah se
- GEval do cheezein fix karta hai: (1) CoT se fixed evaluation steps, (2) top-k token probabilities ka weighted average
- GEval sirf ek improved LLM-as-a-Judge hai — koi bilkul naya concept nahi, bas do smart innovations hain
- Correctness** = duniya ki nazar mein factually sahi (Faithfulness se alag — wo sirf context-groundedness check karta hai)
- Prompt engineering genuinely matter karta hai — generator ke system prompt ko tweak karke hi Completeness aur Style scores improve hue
- Rubric mein explicitly bata do: scoring bands, kya penalize karna hai, kya nahi — isse zyada deterministic results milte hain (re-run pe score barely move karta hai)
- Agla topic: **Safety aur Operations metrics + Regression Testing** (next session)

"GEval ek research technique hai jo naive LLM-as-a-judge ke high-variance problem ko fix karta hai — fixed evaluation steps aur probability-weighted scoring ke through. Ye judgment-based metrics (Correctness, Completeness, Style) ke liye zaroori hai, jahan simple counting kaam nahi karti."

Note: Ye notes RAG Evaluation series ke ek session (Application Quality — GEval) par based hain. Standalone document hai, kisi aur session ke notes se merge nahi kiya gaya hai.